



L5B: Tree Processing & CPS

CS1101S: Programming Methodology

Sanka Rasnayaka

11th September, 2026

Version 1.0 — Built: 2026-09-11 06:23:35 SGT



Announcements

- ▶ Practical Assessment 1 Details - on Canvas



Announcements

- ▶ Practical Assessment 1 Details - on Canvas
- ▶ PA1 practice papers - on SourceAcademy



Announcements

- ▶ Practical Assessment 1 Details - on Canvas
- ▶ PA1 practice papers - on SourceAcademy
- ▶ Source Academy Awards: Beautifully Built.



Announcements

- ▶ Practical Assessment 1 Details - on Canvas
- ▶ PA1 practice papers - on SourceAcademy
- ▶ Source Academy Awards: Beautifully Built.
- ▶ Upcoming contest: The Choreographer



Outline

- ▶ Trees and Tree Processing (2.2.2)
<https://sourceacademy.nus.edu.sg/sicpy/2.2.2>
- ▶ Continuation-Passing Style (CPS)



Review: The Trio

```
def map(f, xs):
    return (None
            if is_none(xs)
            else pair(f(head(xs)), map(f, tail(xs))))

def filter(pred, xs):
    return (None
            if is_none(xs)
            else pair(head(xs), filter(pred, tail(xs)))
               if pred(head(xs))
               else filter(pred, tail(xs)))

def reduce(op, initial, xs):
    return (initial
            if is_none(xs)
            else op(head(xs),
                    reduce(op, initial, tail(xs))))
```



Outline

- ▶ Trees and Tree Processing (2.2.2)
<https://sourceacademy.nus.edu.sg/sicpy/2.2.2>
- ▶ Continuation-Passing Style (CPS)



Trees

- Definition: A **tree** of a certain data type is a llist whose elements are of that data type, or trees of that data type



Trees

- ▶ Definition: A **tree** of a certain data type is a llist whose elements are of that data type, or trees of that data type
- ▶ Examples:



Trees

- ▶ Definition: A **tree** of a certain data type is a llist whose elements are of that data type, or trees of that data type
- ▶ Examples:

```
# trees of numbers  
tree_a = llist(1, 2, 3, 4)
```





Trees

- ▶ Definition: A **tree** of a certain data type is a llist whose elements are of that data type, or trees of that data type
- ▶ Examples:

```
# trees of numbers  
tree_a = llist(1, 2, 3, 4)  
tree_b = llist(llist(1, 2), llist(3, 4))
```



Trees

- ▶ Definition: A **tree** of a certain data type is a llist whose elements are of that data type, or trees of that data type
- ▶ Examples:

```
# trees of numbers  
tree_a = llist(1, 2, 3, 4)  
tree_b = llist(llist(1, 2), llist(3, 4))  
tree_c = None # can be a tree of any type
```



Trees

- ▶ Definition: A **tree** of a certain data type is a llist whose elements are of that data type, or trees of that data type
- ▶ Examples:

```
# trees of numbers
tree_a = llist(1, 2, 3, 4)
tree_b = llist(llist(1, 2), llist(3, 4))
tree_c = None # can be a tree of any type
tree_d = llist(llist(1, 2), None, 3, llist(4, None))
```



Trees

- ▶ Definition: A **tree** of a certain data type is a llist whose elements are of that data type, or trees of that data type
- ▶ Examples:

```
# trees of numbers
tree_a = llist(1, 2, 3, 4)
tree_b = llist(llist(1, 2), llist(3, 4))
tree_c = None # can be a tree of any type
tree_d = llist(llist(1, 2), None, 3, llist(4, None))
```



Trees

- ▶ Definition: A **tree** of a certain data type is a llist whose elements are of that data type, or trees of that data type
- ▶ Examples:

```
# trees of numbers
tree_a = llist(1, 2, 3, 4)
tree_b = llist(llist(1, 2), llist(3, 4))
tree_c = None # can be a tree of any type
tree_d = llist(llist(1, 2), None, 3, llist(4, None))
```

- ▶ Caveat: Cannot consider `None` and `pair` as “certain data type”
 - So, we cannot have trees of `None` and trees of `pairs`



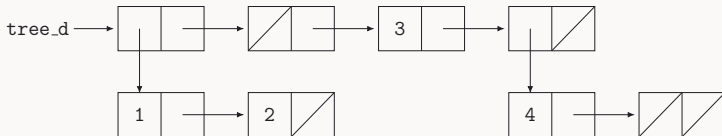
Visualizing Trees

- ▶ Definition: A **tree** of a certain data type is a llist whose elements are of that data type, or trees of that data type

Visualizing Trees

- Definition: A **tree** of a certain data type is a llist whose elements are of that data type, or trees of that data type

```
tree_d = llist(llist(1, 2), None, 3, llist(4, None))
```





Trees in CS

- ▶ Example tree of numbers:

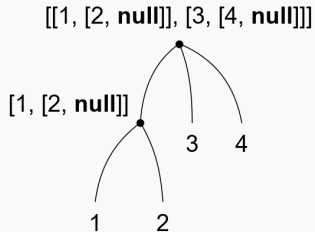
```
tree_c = llist(llist(1, 2), 3, 4)
```



Trees in CS

► Example tree of numbers:

```
tree_c = llist(llist(1, 2), 3, 4)
```

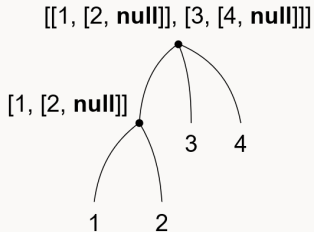




Trees in CS

- Example tree of numbers:

```
tree_c = llist(llist(1, 2), 3, 4)
```





Counting Data Items in a Tree

- ▶ Wanted: `count_data_items(tree)` returns the number of data items in a given tree

```
# example:  
tree_c = llist(llist(1, 2), None, 3, llist(4, None))
```

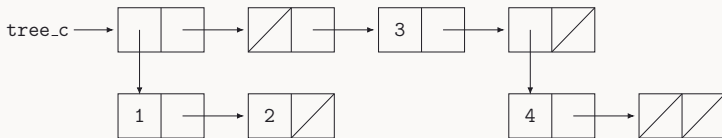


Counting Data Items in a Tree

- ▶ Wanted: `count_data_items(tree)` returns the number of data items in a given tree

example:

```
tree_c = llist(llist(1, 2), None, 3, llist(4, None))
```



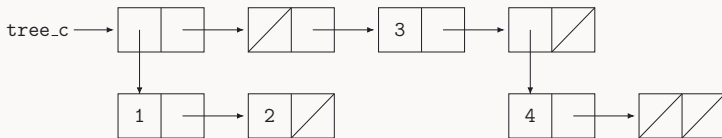


Counting Data Items in a Tree

- ▶ Wanted: `count_data_items(tree)` returns the number of data items in a given tree

example:

```
tree_c = llist(llist(1, 2), None, 3, llist(4, None))
```



```
count_data_items(tree_c)
```

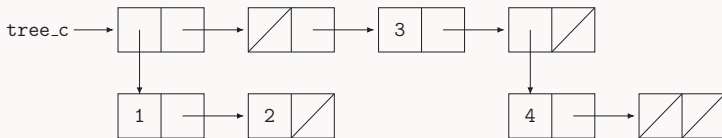


Counting Data Items in a Tree

- ▶ Wanted: `count_data_items(tree)` returns the number of data items in a given tree

example:

```
tree_c = llist(llist(1, 2), None, 3, llist(4, None))
```



```
count_data_items(tree_c)
```

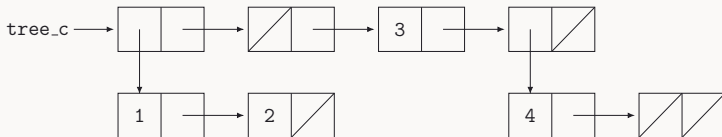
⇒ 4



Counting Data Items in a Tree: The Idea



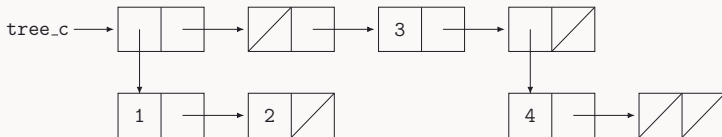
Counting Data Items in a Tree: The Idea



► Idea:



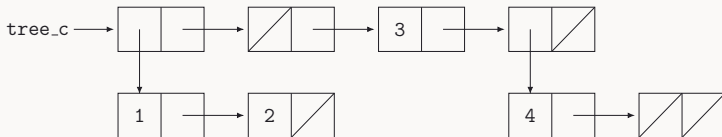
Counting Data Items in a Tree: The Idea



- Idea:
 - Every tree is a llist

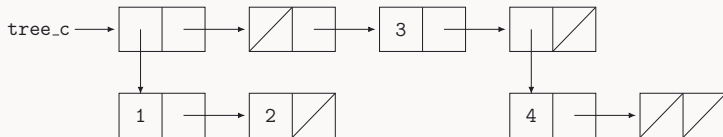


Counting Data Items in a Tree: The Idea



► Idea:

- Every tree is a llist
- If the llist is empty, we return 0

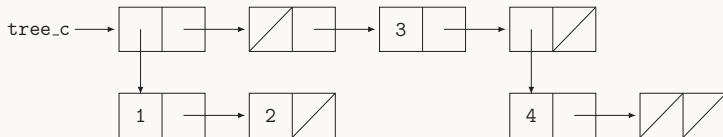


► Idea:

- Every tree is a llist
- If the llist is empty, we return 0
- If the llist is not empty, we add the number of data items of the head to the number of data items of the tail



Counting Data Items in a Tree: The Idea



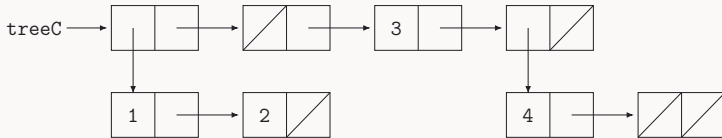
► Idea:

- Every tree is a llist
- If the llist is empty, we return 0
- If the llist is not empty, we add the number of data items of the head to the number of data items of the tail
 - If the head is a tree/llist, we count its data items
 - If it's not a tree/llist, it's a data item and we count 1



Counting Data Items in a Tree: Implementation

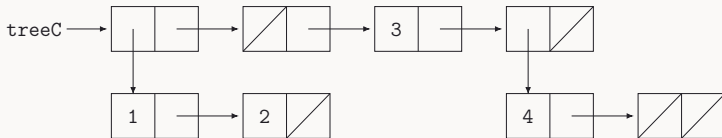
```
def count_data_items(tree):
```





Counting Data Items in a Tree: Implementation

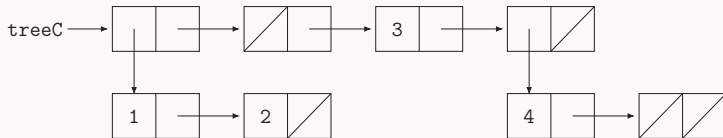
```
def count_data_items(tree):  
    if is_none(tree):  
        return 0
```





Counting Data Items in a Tree: Implementation

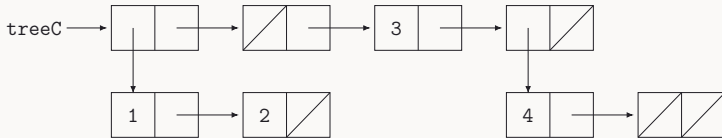
```
def count_data_items(tree):  
    if is_none(tree):  
        return 0  
    elif is_llist(head(tree)):
```





Counting Data Items in a Tree: Implementation

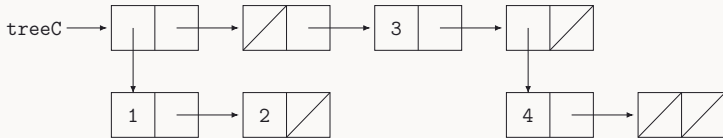
```
def count_data_items(tree):  
    if is_none(tree):  
        return 0  
    elif is_llist(head(tree)):  
  
    else:
```





Counting Data Items in a Tree: Implementation

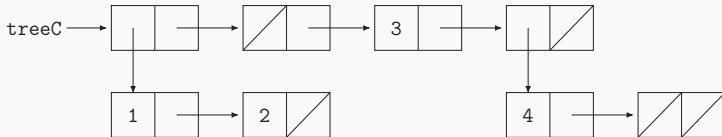
```
def count_data_items(tree):  
    if is_none(tree):  
        return 0  
    elif is_llist(head(tree)):  
        return (count_data_items(head(tree)) +  
  
    else:
```





Counting Data Items in a Tree: Implementation

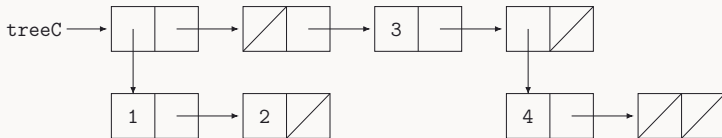
```
def count_data_items(tree):  
    if is_none(tree):  
        return 0  
    elif is_llist(head(tree)):  
        return (count_data_items(head(tree)) +  
                count_data_items(tail(tree)))  
    else:
```





Counting Data Items in a Tree: Implementation

```
def count_data_items(tree):  
    if is_none(tree):  
        return 0  
    elif is_llist(head(tree)):  
        return (count_data_items(head(tree)) +  
                count_data_items(tail(tree)))  
    else:  
        return 1 + count_data_items(tail(tree))
```





Scaling a Tree

- ▶ Wanted: `scale_tree(tree, k)` returns a new tree with the data items of `tree` scaled by the factor `k`



Scaling a Tree

- ▶ Wanted: `scale_tree(tree, k)` returns a new tree with the data items of `tree` scaled by the factor `k`

```
# example:  
tree_c = llist(llist(1, 2), None, 3, llist(4, None))  
scale_tree(tree_c, 10)
```





Scaling a Tree

- ▶ Wanted: `scale_tree(tree, k)` returns a new tree with the data items of `tree` scaled by the factor `k`

```
# example:  
tree_c = llist(llist(1, 2), None, 3, llist(4, None))  
scale_tree(tree_c, 10)
```



⇒ `llist(llist(10, 20), None, 30, llist(40, None))`



Scaling a Tree: The Implementation

- ▶ Recall: A tree is a llist whose elements are data items, or trees



Scaling a Tree: The Implementation

- ▶ Recall: A tree is a llist whose elements are data items, or trees
- ▶ Idea: Map over the llist — if element is a data item, scale element, if not, scale tree



Scaling a Tree: The Implementation

- ▶ Recall: A tree is a llist whose elements are data items, or trees
- ▶ Idea: Map over the llist — if element is a data item, scale element, if not, scale tree
- ▶ Implementation:



Scaling a Tree: The Implementation

- ▶ Recall: A tree is a llist whose elements are data items, or trees
- ▶ Idea: Map over the llist — if element is a data item, scale element, if not, scale tree
- ▶ Implementation:

```
def scale_tree(tree, k):
```





Scaling a Tree: The Implementation

- ▶ Recall: A tree is a llist whose elements are data items, or trees
- ▶ Idea: Map over the llist — if element is a data item, scale element, if not, scale tree
- ▶ Implementation:

```
def scale_tree(tree, k):  
    return map(  

```





Scaling a Tree: The Implementation

- ▶ Recall: A tree is a llist whose elements are data items, or trees
- ▶ Idea: Map over the llist — if element is a data item, scale element, if not, scale tree
- ▶ Implementation:

```
def scale_tree(tree, k):  
    return map(  
        lambda sub_tree:
```





Scaling a Tree: The Implementation

- ▶ Recall: A tree is a llist whose elements are data items, or trees
- ▶ Idea: Map over the llist — if element is a data item, scale element, if not, scale tree
- ▶ Implementation:

```
def scale_tree(tree, k):  
    return map(  
        lambda sub_tree:  
  
        tree)
```



Scaling a Tree: The Implementation

- ▶ Recall: A tree is a llist whose elements are data items, or trees
- ▶ Idea: Map over the llist — if element is a data item, scale element, if not, scale tree
- ▶ Implementation:

```
def scale_tree(tree, k):  
    return map(  
        lambda sub_tree:  
            k * sub_tree  
  
        tree)
```



Scaling a Tree: The Implementation

- ▶ Recall: A tree is a llist whose elements are data items, or trees
- ▶ Idea: Map over the llist — if element is a data item, scale element, if not, scale tree
- ▶ Implementation:

```
def scale_tree(tree, k):  
    return map(  
        lambda sub_tree:  
            k * sub_tree  
            if not is_llist(sub_tree)  
  
        tree)
```



Scaling a Tree: The Implementation

- ▶ Recall: A tree is a llist whose elements are data items, or trees
- ▶ Idea: Map over the llist — if element is a data item, scale element, if not, scale tree
- ▶ Implementation:

```
def scale_tree(tree, k):  
    return map(  
        lambda sub_tree:  
            k * sub_tree  
            if not is_llist(sub_tree)  
            else scale_tree(sub_tree, k),  
        tree)
```



Abstraction: Mapping over Trees

```
def map_tree(f, tree):  
    return map(  
        lambda sub_tree:  
            f(sub_tree)  
            if not is_llist(sub_tree)  
            else map_tree(f, sub_tree),  
        tree)
```



scale_tree **using** map_tree



scale_tree **using** map_tree

```
def scale_tree(tree, k):  
    return map_tree(lambda data_item: data_item * k,  
                    tree)
```





Check-In time!

To tree or not To tree, that is the question!

Look for Check-In I5B in the Source Academy!



Outline

- ▶ Trees and Tree Processing (2.2.2)
<https://sourceacademy.nus.edu.sg/sicpy/2.2.2>
- ▶ Continuation-Passing Style (CPS)



A Closer Look at append



A Closer Look at append

```
def append(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        return pair(head(xs), append(tail(xs), ys))
```





A Closer Look at append

```
def append(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        return pair(head(xs), append(tail(xs), ys))
```

- Can we do this using an iterative process?



Iterative append

- ▶ First attempt:



Iterative append

► First attempt:

```
def append_iter(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        return append_iter(tail(xs), pair(head(xs), ys))
```



Iterative append

► First attempt:

```
def append_iter(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        return append_iter(tail(xs), pair(head(xs), ys))
```

```
append_iter(llist(1, 2, 3), llist(4, 5, 6))
```

Iterative append

► First attempt:

```
def append_iter(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        return append_iter(tail(xs), pair(head(xs), ys))
```

`append_iter(llist(1, 2, 3), llist(4, 5, 6))`
 $\Rightarrow$ `llist(3, 2, 1, 4, 5, 6)`



Iterative append

- ▶ Second attempt (using reverse):



Iterative append

- ▶ Second attempt (using reverse):

```
def append_iter(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        return append_iter(tail(xs), pair(head(xs), ys))  
  
def append(xs, ys):  
    return append_iter(reverse(xs), ys)
```



Iterative append

- ▶ Second attempt (using reverse):

```
def append_iter(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        return append_iter(tail(xs), pair(head(xs), ys))  
  
def append(xs, ys):  
    return append_iter(reverse(xs), ys)
```

```
append(llist(1, 2, 3), llist(4, 5, 6))
```

Iterative append

- ▶ Second attempt (using reverse):

```
def append_iter(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        return append_iter(tail(xs), pair(head(xs), ys))  
  
def append(xs, ys):  
    return append_iter(reverse(xs), ys)
```

```
append(llist(1, 2, 3), llist(4, 5, 6))  
⇒ llist(1, 2, 3, 4, 5, 6)
```



Another Iterative Version of append



Another Iterative Version of append

```
def append(xs, ys): # Recursive process
    if is_none(xs):
        return ys
    else:
        return pair(head(xs), append(tail(xs), ys))
```



Another Iterative Version of append

```
def append(xs, ys): # Recursive process
    if is_none(xs):
        return ys
    else:
        return pair(head(xs), append(tail(xs), ys))

def app(current_xs, ys, c): # Iterative process
```



Another Iterative Version of append

```
def append(xs, ys): # Recursive process
    if is_none(xs):
        return ys
    else:
        return pair(head(xs), append(tail(xs), ys))

def app(current_xs, ys, c): # Iterative process
    if is_none(current_xs):
        return c(ys)
```

Another Iterative Version of append

```
def append(xs, ys): # Recursive process
    if is_none(xs):
        return ys
    else:
        return pair(head(xs), append(tail(xs), ys))

def app(current_xs, ys, c): # Iterative process
    if is_none(current_xs):
        return c(ys)
    else:
```



Another Iterative Version of append

```
def append(xs, ys): # Recursive process
    if is_none(xs):
        return ys
    else:
        return pair(head(xs), append(tail(xs), ys))

def app(current_xs, ys, c): # Iterative process
    if is_none(current_xs):
        return c(ys)
    else:
        return app(tail(current_xs), ys,
                    lambda x: c(pair(head(current_xs), x)))
```



Another Iterative Version of append

```
def append(xs, ys): # Recursive process
    if is_none(xs):
        return ys
    else:
        return pair(head(xs), append(tail(xs), ys))

def app(current_xs, ys, c): # Iterative process
    if is_none(current_xs):
        return c(ys)
    else:
        return app(tail(current_xs), ys,
                    lambda x: c(pair(head(current_xs), x)))

def append_iter(xs, ys):
    return app(xs, ys, lambda x: x)
```



Continuation-Passing Style



Continuation-Passing Style

```
def app(current_xs, ys, c): # Iterative process
    if is_none(current_xs):
        return c(ys)
    else:
        return app(tail(current_xs), ys,
                    lambda x: c(pair(head(current_xs), x)))

def append_iter(xs, ys):
    return app(xs, ys, lambda x: x)
```



Continuation-Passing Style

```
def app(current_xs, ys, c): # Iterative process
    if is_none(current_xs):
        return c(ys)
    else:
        return app(tail(current_xs), ys,
                    lambda x: c(pair(head(current_xs), x)))

def append_iter(xs, ys):
    return app(xs, ys, lambda x: x)
```

- Programming Pattern: CPS



Continuation-Passing Style

```
def app(current_xs, ys, c): # Iterative process
    if is_none(current_xs):
        return c(ys)
    else:
        return app(tail(current_xs), ys,
                    lambda x: c(pair(head(current_xs), x)))

def append_iter(xs, ys):
    return app(xs, ys, lambda x: x)
```

- ▶ Programming Pattern: CPS
 - Passing the deferred operation as a function in an extra argument is called “Continuation-Passing Style” (CPS)



Continuation-Passing Style

```
def app(current_xs, ys, c): # Iterative process
    if is_none(current_xs):
        return c(ys)
    else:
        return app(tail(current_xs), ys,
                    lambda x: c(pair(head(current_xs), x)))

def append_iter(xs, ys):
    return app(xs, ys, lambda x: x)
```

► Programming Pattern: CPS

- Passing the deferred operation as a function in an extra argument is called “Continuation-Passing Style” (CPS)
- We can convert any recursive function this way!



CPS Example: Factorial



CPS Example: Factorial

```
def factorial(n): # Recursive process
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)
```





CPS Example: Factorial

```
def factorial(n): # Recursive process
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)
```

```
def fac(n, c): # Iterative process
    return (c(1)
            if n == 1
            else fac(n - 1, lambda x: c(n * x)))

def factorial_iter(n):
    return fac(n, lambda x: x)
```



Analysing CPS Factorial

```
def fac(n, c):  
    return (c(1) if n == 1  
            else fac(n - 1, lambda x: c(n * x)))  
  
def factorial_iter(n):  
    return fac(n, lambda x: x)
```

factorial_iter(3)



Analysing CPS Factorial

```
def fac(n, c):  
    return (c(1) if n == 1  
            else fac(n - 1, lambda x: c(n * x)))  
  
def factorial_iter(n):  
    return fac(n, lambda x: x)
```

factorial_iter(3)

fac(3, lambda x: x)



Analysing CPS Factorial

```
def fac(n, c):  
    return (c(1) if n == 1  
            else fac(n - 1, lambda x: c(n * x)))  
  
def factorial_iter(n):  
    return fac(n, lambda x: x)
```

factorial_iter(3)

fac(3, lambda x: x)

fac(2, lambda x: (lambda x: x)(3 * x))



Analysing CPS Factorial

```
def fac(n, c):  
    return (c(1) if n == 1  
            else fac(n - 1, lambda x: c(n * x)))  
  
def factorial_iter(n):  
    return fac(n, lambda x: x)
```

factorial_iter(3)

fac(3, lambda x: x)

fac(2, lambda x: (lambda x: x)(3 * x))

fac(1, lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x))



Analysing CPS Factorial

```
def fac(n, c):  
    return (c(1) if n == 1  
            else fac(n - 1, lambda x: c(n * x)))  
  
def factorial_iter(n):  
    return fac(n, lambda x: x)
```

factorial_iter(3)

fac(3, lambda x: x)

fac(2, lambda x: (lambda x: x)(3 * x))

fac(1, lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x))

lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x)(1)



Analysing CPS Factorial

```
def fac(n, c):  
    return (c(1) if n == 1  
            else fac(n - 1, lambda x: c(n * x)))  
  
def factorial_iter(n):  
    return fac(n, lambda x: x)
```

factorial_iter(3)

fac(3, lambda x: x)

fac(2, lambda x: (lambda x: x)(3 * x))

fac(1, lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x))

lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x)(1)

(lambda x: (lambda x: x)(3 * x))(2 * 1)



Analysing CPS Factorial

```
def fac(n, c):  
    return (c(1) if n == 1  
            else fac(n - 1, lambda x: c(n * x)))  
  
def factorial_iter(n):  
    return fac(n, lambda x: x)
```

factorial_iter(3)

fac(3, lambda x: x)

fac(2, lambda x: (lambda x: x)(3 * x))

fac(1, lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x))

lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x)(1)

(lambda x: (lambda x: x)(3 * x))(2 * 1)

(lambda x: (lambda x: x)(3 * x))(2)



Analysing CPS Factorial

```
def fac(n, c):  
    return (c(1) if n == 1  
            else fac(n - 1, lambda x: c(n * x)))  
  
def factorial_iter(n):  
    return fac(n, lambda x: x)
```

factorial_iter(3)

fac(3, lambda x: x)

fac(2, lambda x: (lambda x: x)(3 * x))

fac(1, lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x))

lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x)(1)

(lambda x: (lambda x: x)(3 * x))(2 * 1)

(lambda x: (lambda x: x)(3 * x))(2)

(lambda x: x)(3 * 2)



Analysing CPS Factorial

```
def fac(n, c):  
    return (c(1) if n == 1  
            else fac(n - 1, lambda x: c(n * x)))  
  
def factorial_iter(n):  
    return fac(n, lambda x: x)
```

factorial_iter(3)

fac(3, lambda x: x)

fac(2, lambda x: (lambda x: x)(3 * x))

fac(1, lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x))

lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x)(1)

(lambda x: (lambda x: x)(3 * x))(2 * 1)

(lambda x: (lambda x: x)(3 * x))(2)

(lambda x: x)(3 * 2)

(lambda x: x)(6)



Analysing CPS Factorial

```
def fac(n, c):  
    return (c(1) if n == 1  
            else fac(n - 1, lambda x: c(n * x)))  
  
def factorial_iter(n):  
    return fac(n, lambda x: x)
```

factorial_iter(3)

fac(3, lambda x: x)

fac(2, lambda x: (lambda x: x)(3 * x))

fac(1, lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x))

lambda x: (lambda x: (lambda x: x)(3 * x))(2 * x)(1)

(lambda x: (lambda x: x)(3 * x))(2 * 1)

(lambda x: (lambda x: x)(3 * x))(2)

(lambda x: x)(3 * 2)

(lambda x: x)(6)

6



Refining the Space Requirement definition

- ▶ CPS functions creates more functions as they execute



Refining the Space Requirement definition

- ▶ CPS functions creates more functions as they execute
- ▶ The number of functions created depends on input!



Refining the Space Requirement definition

- ▶ CPS functions creates more functions as they execute
- ▶ The number of functions created depends on input!
- ▶ Space in memory is required to create these functions!



Refining the Space Requirement definition

- ▶ CPS functions creates more functions as they execute
- ▶ The number of functions created depends on input!
- ▶ Space in memory is required to create these functions!
- ▶ Space Complexity depends on,
 - Deferred Operations
 - Compound Data values (pairs)
 - Function values created



Refining the Space Requirement definition

- ▶ CPS functions creates more functions as they execute
- ▶ The number of functions created depends on input!
- ▶ Space in memory is required to create these functions!
- ▶ Space Complexity depends on,
 - Deferred Operations
 - Compound Data values (pairs)
 - Function values created
- ▶ Are all these Spaces equal?



Refining the Space Requirement definition

- ▶ CPS functions creates more functions as they execute
- ▶ The number of functions created depends on input!
- ▶ Space in memory is required to create these functions!
- ▶ Space Complexity depends on,
 - Deferred Operations
 - Compound Data values (pairs)
 - Function values created
- ▶ Are all these Spaces equal?
 - Deferred operations require space in “control”
 - Data & functions require space in “environment”
 - more in **Unit 3!** :)



Another CPS Example

- Recall fractal from Lecture L3A

```
def fractal_3(rune, n):  
    if n == 1:  
        return rune  
    else:  
        f = fractal_3(rune, n - 1)  
        return beside(rune, stack(f, f))
```





Another CPS Example

```
from rune import beside, stack
```





Another CPS Example

```
from rune import beside, stack

def fractal_5(rune, n): # Recursive process
    if n == 1:
        return rune
    else:
        return beside(rune, fractal_5(stack(rune, rune),
n - 1))
```



Another CPS Example

```
from rune import beside, stack

def fractal_5(rune, n): # Recursive process
    if n == 1:
        return rune
    else:
        return beside(rune, fractal_5(stack(rune, rune),
                                          n - 1))

def frac(rune, n, c): # Iterative process
    if n == 1:
        return c(rune)
    else:
        return frac(stack(rune, rune), n - 1,
                      lambda res: c(beside(rune, res)))

def fractal_5_iter(rune, n):
    return frac(rune, n, lambda rune: rune)
```



Summary

- ▶ Trees, tree processing, and higher-order tree processing
- ▶ Continuation-Passing Style can be used to turn any function iterative